

- B.PREDICT 基本設計書
 - 1. システム全体像
 - 1.1 構成方針
 - 1.2 構成図
 - 1.3 データフロー
 - 2. サブシステム設計
 - 2.1 データ取得サブシステム (Python)
 - 2.2 特微量サブシステム (Python)
 - 2.3 モデルサブシステム (Python)
 - 2.4 API サブシステム (Workers / TypeScript)
 - 2.5 フロントエンドサブシステム (Next.js / TypeScript)
 - 3. データモデル設計
 - 3.1 テーブル分類
 - 3.2 重要な設計判断
 - 4. 処理設計
 - 4.1 バッチジョブ一覧
 - 4.2 日次取り込みの流れ
 - 4.3 エラー処理方針
 - 4.4 幂等性
 - 4.5 更新遅延の判定
 - 5. 画面設計
 - 5.1 画面遷移
 - 5.2 主要画面の構成
 - 5.3 表示ルール
 - 6. デザインシステム
 - 6.1 デザイントークン
 - 6.2 タイポグラフィ
 - 6.3 レイアウト原則
 - 6.4 チャート
 - 7. インフラ・デブロイ設計
 - 7.1 リポジトリ構成
 - 7.2 CI
 - 7.3 Actions のセキュリティ
 - 7.4 シークレット管理
 - 7.5 無料枠の監視
 - 8. 品質保証
 - 8.1 テスト方針
 - 8.2 リーク検証 (最重要)
 - 8.3 その他の必須テスト
 - 8.4 運用監視
 - 9. 未決事項
 - 9.1 実装前に実測が必要な項目

B.PREDICT 基本設計書

項目	内容
版数	1.3
作成日	2026-09-19
改訂	v1.1: 9領域レビューの指摘を反映 / v1.2: 個人スタッツをフルボックススコアに拡張 / v1.3: 実装前検証の結果を反映 (学習スナップショット、モデル構成、静的生成範囲、Next.js 16、ルーティング、CI)
上位文書	docs/requirements.md
下位文書	docs/design-detail.md

1. システム全体像

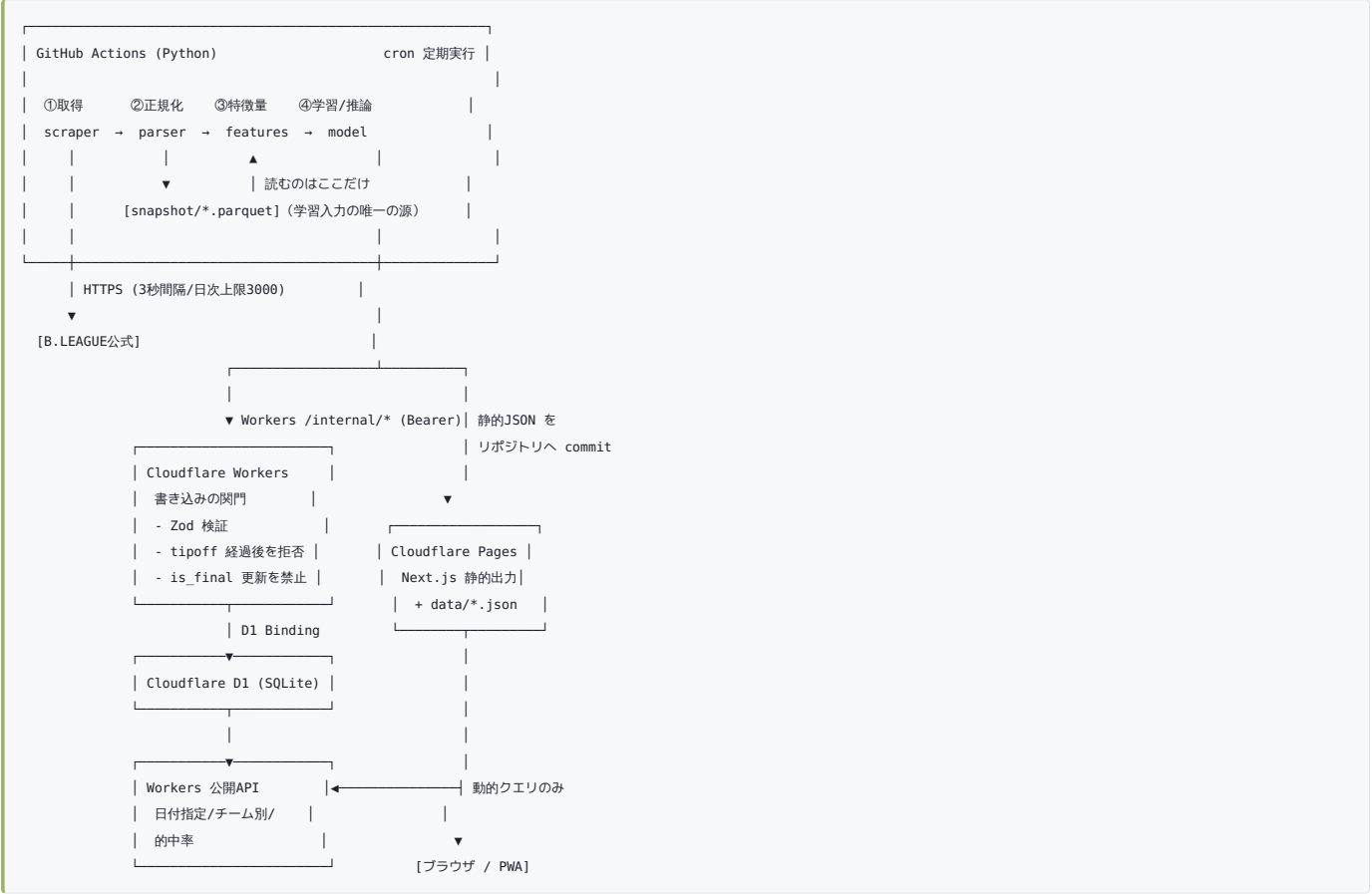
1.1 構成方針

3つの実行環境に役割を分離する。

環境	役割	言語
GitHub Actions	データ取得、特徴量生成、学習、推論、静的JSONの書き出し	Python
Cloudflare Workers	書き込みの関門、動的クエリの配信	TypeScript
Cloudflare Pages	画面の描画、予測データの静的配信	TypeScript (Next.js 静的出力)

重い処理を GitHub Actions に閉じ込め、**主要導線は Workers も D1 も経由させない**。これにより無料枠の消費を構造的にゼロに近づけ、枠が枯渇しても当日の予測が表示され続ける状態を作る。

1.2 構成図



重要な設計判断

- D1 への書き込みは Workers の /internal/* 経由に一本化する。** D1 REST API を直接叩く経路は設けない。REST 直叩きを許すと、Zod 検証・認可・「tipoff 経過後は拒否」というガードがすべて迂回可能になり、予測の不変性という中核価値を守る関門が存在しなくなる。
- 今日の予測・試合一覧は静的JSONで配信する。** バッチが `web/public/data/*.json` を書き出し、Pages の静的アセットとして配信する。Workers リクエストと D1 読取を消費しない。
- ISR を使わない。** Cloudflare Pages 上の Next.js は ISR が成立しない（要件7章）。
- バッチは D1 から読まない。学習・特徴量生成は自前のスナップショットを参照する。** バッチが取得・正規化したデータは、Workers 経由で D1 に書き込むのと同時に、リポジトリ配下の Parquet スナップショット（`batch/snapshot/`）にも書き出す。特徴量生成と学習はこのスナップショットのみを読む。
なぜ D1 からの一括読み出し経路を設けないのか。 D1 は「書き込み先」であって「学習データの供給元」ではない。一括エクスポート経路を残すと、(a) 読取行数の上限（500万行per日）を学習が消費する構造が残り、(b) 学習時のデータ取得が Cloudflare の可用性に依存し、(c) 「バッチは D1 を読んでよい」という例外が一度でも許されると、逐次クエリへの退行を設計で止められなくなる。**スナップショットが唯一の学習入力であると決めれば**、この3つが同時に消える。
スナップショットは決定論的に再生成でき、D1 の内容と一致することをテストで保証する（8.3）。

1.3 データフロー

試合前日以前（日次 06:00 JST）

前日結果取得 → 結果照合 → Elo再計算 → 特徴量生成 → 推論
→ predictions 追記 → 静的JSON書き出し → Pages デプロイ

試合当日（固定cron 3スロット。各スロットで tipoff > now+30分 の試合に限定）

エントリー取得 → 出場者確定判定 → 再推論
→ predictions 追記（旧行を is_active=0）→ 静的JSON更新

試合開始時刻

freeze (Workers の Cron Trigger が毎時実行、または読み取り時の遅延判定)
以後 /internal/predictions は当該試合を 409 で拒否

翌日 06:00 JST

結果取得 → 確定予測と照合 → prediction_results 登録 → 集計テーブル更新

月次 / シーズン開始時

全データ再学習 → 同一評価ウィンドウで検証 → 基準を満たせば有効化

2. サブシステム設計

2.1 データ取得サブシステム（Python）

batch/

└─ scraper/外部サイトへのアクセス。HTTPのみを担当

├─ client.pyレート制限・日次上限・サイズ上限・robots照合

├─ schedule.py / boxscore.py / entry.py

└─ parser/HTML → 構造化データ。サイト構造への依存をここに閉じる

├─ schedule_parser.py / boxscore_parser.py / entry_parser.py

├─ validators.py値域検証

└─ loader/Workers /internal/* への書き込み

└─ models.py

設計原則: サイト構造の変更は `parser/` のみの修正で吸収できること。 `scraper/` は HTTP の作法だけを持ち、 `parser/` は HTML の解釈だけを持つ。

アクセス制御

項目	値
リクエスト間隔	最低3秒（ジッタ ±1秒）
並列度	1
日次リクエスト上限	3,000（クライアント側で強制）
レスポンスサイズ上限	5MB（超過で中止）
タイムアウト	30秒
中止条件	429 / 503 受信時はジョブ全体を即時中止
事前確認	<code>robots.txt</code> と利用規約のハッシュ照合。変更検知でジョブ中止
User-Agent	<code><name>/1.0 (+https://<domain>/about)</code>

値域検証（`parser/validators.py`）

スクレイピングは入力が信用できないパイプラインである。必須項目の有無だけでなく値域も検証し、範囲外は当該試合をスキップする。異常値が Elo 計算に流れ込むとレーティング全体が破壊され、冪等な再実行では復旧しない。

項目	範囲・条件
<code>pts</code>	0-250
<code>minutes</code>	0-60
シュート	<code>fg2m <= fg2a</code> 、 <code>fg3m <= fg3a</code> 、 <code>ftm <= fta</code>
<code>pf</code>	0-6
<code>attendance</code>	<code><= capacity × 1.2</code>
<code>possessions</code>	50-120
恒等式	<code>pts = fg2m×2 + fg3m×3 + ftm</code> 、 <code>fga = fg2a + fg3a</code> 、 <code>reb = oreb + dreb</code> 、チーム集計 = 所属選手の合計

2.2 特徴量サブシステム（Python）

batch/features/	
└─ base.py	基底クラスと登録機構
└─ team_strength.py / schedule_ctx.py / player.py / venue.py	
└─ dataset.py	スナップショットの読み込みとメモリ内データセット
└─ builder.py	as_of 時点の特徴量ベクトル生成
batch/snapshot/	
学習入力 (Parquet)。バッチが書き、バッチが読む	
└─ games.parquet / team_games.parquet / team_game_stats.parquet	
└─ player_game_stats.parquet / game_entries.parquet	
└─ clubs.parquet / players.parquet / venues.parquet / seasons.parquet	
└─ club_seasons.parquet / player_seasons.parquet / venue_revisions.parquet	
└─ MANIFEST.json	各ファイルの行数・SHA256・生成時刻・最新 finished_at

中核となる設計: すべての特徴量関数は `as_of` を引数に取り、**その時刻以前に終了した試合の情報のみ**を参照する。絞り込みは `tipoff_at` ではなく `finished_at` で行う（`tipoff_at` で絞ると、17:05開始・19:05終了の試合が19:05開始の試合の確定情報として混入する）。

特徴量生成と学習は D1 を読まない。 入力はバッチ自身が保持する Parquet スナップショットのみとする。試合8,000件・選手スタッツ20万行は Parquet で数十MB、メモリ上でも数十MB に収まるため、全量をロードして結合・フィルタする。

この方針はリーク防止と無料枠維持の両方に効く。逐次クエリ方式では、`games` の全件スキャンを試合ごとに繰り返すため **1日150万～1,450万行**（上限500万行per日）に達し、学習時は億行規模になる。一括エクスポート方式でも D1 の読取枠を消費するうえ、学習が Cloudflare の可用性に依存する。**スナップショットを唯一の入力と決めることで、D1 の読取行数は「公開APIの動的クエリ分」だけになる。**

スナップショットの整合性保証

項目	内容
書き出し	<code>daily_ingest</code> のステップ3（正規化直後、D1 書き込みと同一データから）
形式	Parquet（列指向・圧縮。CSV よりロードが速く型が保存される）
コミット	リボジトリに含める。生HTMLではなく事実データの構造化結果であるため4.5.2 に抵触しない
検証	<code>MANIFEST.json</code> に各ファイルの行数と SHA256 を記録。学習前に照合する
再現性	スナップショットは D1 から決定論的に再生成できる（ <code>scripts/rebuild_snapshot.py</code> ）。ただし これは復旧手段であり、日常の学習経路ではない
一致テスト	<code>test_snapshot_matches_d1</code> （CI では D1 ローカルに対して実行）

スナップショットが破損・欠損した場合、学習を中止して現行モデルを継続使用する（推論は可能。推論に必要な特徴量も同じスナップショットから作るため、欠損時は前回の静的JSONを維持する）。

2.3 モデルサブシステム (Python)

batch/model/	
└─ dataset.py	学習行列の構築（時間減衰重み・観客制限フラグを含む）
└─ train_winner.py	勝敗（経路A）
└─ train_score.py	得点差と合計得点（経路B の σ 推定を含む）
└─ train_team_rates.py	チームの試投数・成功率（整合化の目標値）
└─ train_player.py	出場確率 → 出場時間 → スタッツ
└─ reconcile.py	整合化（ロジット空間シフト。要件 6.8.5）
└─ calibrate.py	校正（必要性の判定を含む）
└─ explain.py	SHAP のグループ集約
└─ evaluate.py	walk-forward 検証とベースライン比較
└─ registry.py	モデルの保存・読み出し・バージョン管理

モデル構成

モデル	種別	出力
Winner	LightGBM 二値分類	ホーム勝利確率（経路A）
Margin	LightGBM 回帰	得点差（ホーム－アウェイ）。 残差 σ から勝率も導出する（経路B）
Total	LightGBM 回帰	合計得点
TeamRates	LightGBM 回帰 × 14	チームの試投数・成功率・その他カウント （選手予測の整合化目標）
PlayerAvail	LightGBM 二値分類	選手の出場確率
PlayerMinutes	LightGBM 回帰	出場時間（出場する場合）
PlayerRates	LightGBM 回帰 × 14	カウント11項目の単位時間レート＋成功率3項目

予想スコアは $\text{home} = (\text{total} + \text{margin}) / 2$ 、 $\text{away} = (\text{total} - \text{margin}) / 2$ で導出する。各チーム得点を独立に回帰すると、勝率と矛盾した出力が生じ、誤差の分散も過大になる。

勝率は2経路とも実装し、Phase 0（P0-11）で比較して採用経路を決める。

経路	式	長所	短所
A: Winner 直接	<code>p = Winner(x)</code>	logloss を直接最適化するため較正が良い	Margin と符号が食い違いうる
B: Margin から導出	<code>p = $\Phi(\text{margin} / \sigma)$</code> 、 σ は残差標準偏差（実測）	勝率と予想スコアが定義上一致する	残差が正規でない場合に較正が崩れる

判定は同一ウィンドウの Brier と ECE で行い、B が A より Brier で 0.003 以上劣る場合のみ A を採用する（要件 6.1）。採用経路は `model_versions.win_prob_source` に記録する。両方を毎回学習するため、チームモデルの学習時間は約1.2倍になる。

TeamRates を新設する理由。選手予測の整合化（要件 6.8.5）にはチーム側の目標値が必要だが、旧版は「直近N試合平均 × ベース比」という素朴な生成しか決めていなかった。各項目を独立に生成すると**約2%の確率で「FT成功数 > FT試投数」のような実行不能な目標**が生じ、整合化が原理的に成立しない。チーム側も選手側と同じ「試投数 + 成功率」の構造で予測すれば、制約が構造的に保証される。

個人スタッツの4段構成

第1段	PlayerAvail	P(出場)
第2段	PlayerMinutes	E[出場時間 出場]
第3段	PlayerRates	カウント11項目（2FGA/3FGA/FTA/OR/DR/AS/T0/ST/BS/F/FD）を 単位時間あたりレートで、 成功率3項目（2FG%/3FG%/FT%）を率のまま
第4段	Reconciliation	チーム予測への整合化（計算のみ）

シュートは「**試投数 + 成功率**」で予測し、**成功数は導出する**。成功数と試投数を独立に予測すると、整合化の段階で `成功数 > 試投数` が生じ、それを防ぐためのクリップが収束を壊す（実測で失敗ケースの73%が FT 成功率で発生）。試投数と成功率の組で持てば、成功率をロジット空間でシフトするだけで制約が自動的に満たされる。

モデルが出す成功率は「この選手のこの試合における期待成功率」という連続量であり、**画面に出す `FGM / FGA` とは別物**である。表示する率は必ず導出値で、分数と併記する。`FGM` は `2FGM + 3FGM` として導出し、独立に予測しない（恒等式が崩れる）。

成功率3項目は per-minute にしない。率は時間当たりの量ではない。目的変数は過去試合の成功率、学習重みは試投数とし、`(made + k × prior) / (attempted + k)` でシュリンクした値を使う。出力は `clip(0.01, 0.99)`（ロジット変換の定義域確保）。

+/- を予測しない。チームの結果を個人に帰属させた指標で1試合の分散が極端に大きく、MAE は 9〜12点規模になる。しかも「チーム得点差予測 × 出場時間シェア」以上の情報を持たない。実績のみ表示する。

整合化（第4段）が必須である理由

選手ごとに独立予測した値をそのまま表示すると、**5人の得点合計がチームの予想スコアと一致しない**。同じ画面に矛盾した数字が並ぶ。期待出場時間を200分に正規化し、試投数をチーム目標へ比例スケールし、成功率を**ロジット空間で一律シフト**してチーム成功数に合わせ、最後に得点を恒等式で導出する。手順は docs/requirements.md 6.8.5 に定義する。

単純な倍率スケールは使わない（検証済み）。成功率を `[0, 1]` の範囲で一律に掛け算すると、1.0 に張り付いた選手が出た時点で合計を目標に合わせられず振動する。1,500ケースの実測では、2 回反復後の項目誤差の最大が 24.7%、得点誤差が 4.42点に達した。ロジット空間シフトに変更した結果、**得点誤差ゼロ・制約違反ゼロ・成功率が [0,1] を出たケースゼロ**になった。

チーム目標も制約を満たしていなければならない。各項目を独立に生成すると約2%の確率で実行不能な目標（`FT成功数 > FT試投数`）が生まれ、この状態では整合化は原理的に成立しない。TeamRates が「試投数 + 成功率」を出すことで構造的に保証する（3.2）。

検証方式: シーズン単位の walk-forward。各 fold で `train = seasons[:i-1] / valid (early stopping用) = seasons[i-1] / test = seasons[i]` とし、test に一切触れない。ランダムシャッフル分割は禁止。fold 数には上限を設ける（直近5シーズンのみ評価）。設けないとシーズンが増えるたびに学習時間が単調増加する。

最終モデル: 各 fold の `best_iteration` の中央値を `num_boost_round` に固定し、early stopping なして全データを学習する。

再現性: `seed / bagging_seed / feature_fraction_seed / deterministic` を固定し、`model_versions.params` に記録する。固定しないと Brier が ±0.003 ぶれ、採用判定が乱数に支配される。

モデルの保管: LightGBM の `save_model()` はテキスト形式で数百KB〜数MBに収まるため、`model_versions.artifact_text` として **D1 に格納**する。GitHub Actions の artifact は90日で失効し、別ワークフローからの取得にも run-id 解決が必要なため、日次推論の依存先として成立しない。

ただし D1 の1行上限は 2,000,000 バイトであり、artifact は常にその内側とは限らない。テキストモデル書式からの推定は以下のとおり。

num_leaves	木の本数	推定サイズ	2MB 上限に対して
7（設計値）	300〜800	0.27〜0.70 MB	13〜35%
7	2,000	1.75 MB	88%（危険域）
15	1,500	2.56 MB	128%（超過）

したがって次を設計に組み込む。

- 登録時に **1.5MB 上限チェック**を行い、**超えたら登録を拒否して現行モデルを継続使用する**（受け入れ基準 A-18）
- `num_boost_round` に上限（1,200）を設ける。early stopping が効かず上限まで回るケースを防ぐ
- `num_leaves` を 7 から増やす変更は、artifact サイズの再測定とセットでのみ許可する
- 実サイズは Phase 0（P0-12）で測定する

上限に当たった場合の退避手段は gzip + base64（テキストモデルは圧縮率が高く 1/5 程度になる）。v1 では実装せず、P0-12 の結果で必要と判明した場合に追加する。

2.4 API サブシステム（Workers / TypeScript）

```
api/
├─ src/
│  ├─ index.ts
│  ├─ routes/
│  │  ├─ public/    games / teams / accuracy / health
│  │  └─ internal/  書き込み系 (Bearer + tipoff ガード)
│  ├─ repositories/ D1 クエリ (必ず prepare().bind())
│  ├─ schemas/       Zod (型だけでなく値域も定義)
│  └─ middleware/    認証・キャッシュ・CORS・レート制限
└─ wrangler.toml
```

役割: 書き込みの関門と、静的JSONで賄えない動的クエリの配信。今日の予測は Pages の静的アセットが担うため、Workers を通らない。

書き込みの関門として必ず行うこと

1. Bearer トークンの定数時間比較
2. Zod による型と値域の検証
3. `tipoff_at <= now` の試合への予測書き込みを 409 で拒否
4. `is_final = 1` の行を UPDATE / DELETE しない (DB制約でも二重化)
5. 非活性化と挿入を単一の `batch()` にまとめる (D1 にはリクエストを跨ぐトランザクションがない)

キャッシュ: エッジとブラウザで TTL を分離する (`s-maxage` と `max-age`)。 `max-age` を長く取るとブラウザキャッシュを消せず、ページしても届かない。Workers の Cache API はコロ単位で、 `*.pages.dev` ではゾーン/ページも使えないため、「**ページする**」という**到達不能な手段に依存しない設計**にする。キャッシュキーにデータ世代 (最終取り込み時刻) を含めれば、新しい世代が出た時点で別キーになり、ページ自体が不要になる。

保護: CORS はブラウザ内の制限でありサーバからのアクセスを防げない。レート制限の代替にしない。Cloudflare WAF のレートリミットルール (無料枠1ルール、独自ドメインが前提) を `/api/v1/*` に設定する。

2.5 フロントエンドサブシステム (Next.js / TypeScript)

```
web/
├─ app/
│  ├─ page.tsx                今日の予測 (静的JSONを読む)
│  ├─ schedule/[date]/page.tsx 日付別 ([id] との衝突回避のため別階層)
│  ├─ games/[id]/page.tsx     試合詳細 (試合前/試合後で構成が変わる)
│  ├─ results/page.tsx        結果
│  ├─ teams/[slug]/page.tsx   チーム別
│  ├─ accuracy/page.tsx       的中率
│  └─ about/page.tsx          免責・出典・個人情報の取扱い
├─ public/
│  ├─ data/                   パッチが書き出す予測JSON
│  └─ _headers                 セキュリティヘッダ
├─ components/
│  ├─ prediction/ charts/ ui/
└─ styles/tokens.css
```

描画戦略

画面	方式	データ源
今日の予測	静的出力	<code>public/data/today.json</code> (クライアントで読む)
日付別	静的出力	過去日はビルド時、当日は静的JSON
試合詳細	静的出力 (確定済み) + クライアント fetch (未実施)	—
結果・チーム別・的中率	静的出力 + クライアント fetch	Workers API

ISR を使わない。 `generateStaticParams` は**直近5シーズンの範囲内の日付・試合のみ**を生成し、範囲外は `notFound()` で即座に404にする。日付が自由入力のままだと URL 空間が無限になり、クローラの総当たりで無料枠が枯渇する。

静的生成の範囲を直近5シーズンに限定する。 Cloudflare Pages の1デプロイあたりファイル数上限は 20,000 で、1ルートにつき HTML と RSC ペイロードの2ファイルが生成される。

範囲	推定ファイル数	上限に対して
全10シーズン	20,184	101% (超過)
直近5シーズン (採用)	10,184	51%
直近3シーズン	6,184	31%

5シーズンより古いデータは Workers API からのクライアント fetch で表示する。**CI で `out/` のファイル数を数え、18,000 を超えたら fail させる** (A-14) 。1ルートあたりのファイル数は Next.js のバージョンで変わるため、この検査を省略しない。

Next.js 16 に伴う実装上の注意

項目	内容
params / searchParams	Promise になった。 <code>const { date } = await params</code> と書く。同期アクセスは型エラー
next lint	削除された。 ESLint は CLI (<code>eslint .</code>) を直接呼ぶ。 <code>next build</code> はリントを実行しない
ESLint 設定	FlatConfig (<code>eslint.config.mjs</code>) 。 <code>.eslintrc.json</code> は使わない
Node.js	20.9 以上が必須
TypeScript	5.1 以上が必須
Turbopack	既定のバンドラ。静的出力 (<code>output: 'export'</code>) との組み合わせを Phase 0 (P0-14) で確認する

3. データモデル設計

3.1 テーブル分類

分類	テーブル	性質
マスタ (恒久)	clubs players venues seasons	不変の識別子
マスタ (断面)	club_seasons player_seasons venue_revisions	年度で変わる属性
名寄せ	club_source_ids venue_source_keys	公式サイトIDとの対応
ファクト	games team_games team_game_stats player_game_stats game_entries	試合ごとに増える
派生	team_ratings	バッチが生成
予測	predictions player_predictions prediction_reasons	予測の記録
評価	model_versions prediction_results accuracy_summary	精度の記録
運用	ingestion_logs	ジョブ実行履歴

3.2 重要な設計判断

恒久エンティティと断面の分離

時間で変化するもの (クラブの名称・リーグ・本拠地、選手の所属、会場名・収容人数) を単一行の属性として持たない。

- clubs (恒久) / club_seasons (年度断面) / club_source_ids (公式IDの名寄せ)
- players (恒久) / player_seasons (年度断面)
- venues (恒久) / venue_revisions (改称・収容人数変更の履歴)

旧版の teams.legacy_ids (JSON配列) 方式は成立しない。JSON列は SQL で JOIN できず、チーム別集計・Elo・勝率のすべてがインデックスの効かない全展開を通る。また旧IDのまま保存すると team_ratings.team_id が分断され、**旧B1の Elo を新リーグへ引き継げず、コールドスタート対策が実装不能になる。**

取り込み時に club_source_ids で club_id に解決してから保存する。旧IDのまま保存しない。

チーム視点の試合行

team_games (team_id / opponent_id / is_home / result / margin) を持つ。games 単体だと「そのチームが出場した試合」は home_team_id = ? OR away_team_id = ? になり、OR の両辺にインデックスがないと全件スキャンになる。日程系の特徴量はほぼすべてこの経路を通るため、影響が大きい。

予測の世代管理

predictions は**追記のみ**。再推論では旧行を is_active = 0 にし、新しい行を追加する。

- UNIQUE (game_id, model_version, revision) — revision は世代通番
- is_active = 1 の単一性は部分ユニークインデックスで担保する
- is_final = 1 の行は更新・削除を DB 制約で禁止する

旧版の UNIQUE (game_id, model_version, as_of) は成立しない。as_of を tipoff_at 固定にすると、1日3回の再推論がすべて同一キーに衝突し、置換が起きて世代管理が空振りする。「**後から予測を書き換えられない**」という主張が制約自身によって否定される。

as_of と data_as_of の分離

列	意味
as_of	特徴量が参照してよい情報の上限時刻 (= その試合の tipoff_at)
data_as_of	推論実行時点で DB に入っていた最新試合の終了時刻

本番の日次推論は最大7日前の情報で予測するが、バックテストでは全試合が揃っているため開始直前の情報が使える。data_as_of を記録しないと、この差 (検証スコアの体系的な過大評価) を後から検証できない。

日程変更と結果訂正

games の主キーは公式サイトの試合IDとする。自然キー (season_id, game_date, home_team_id, away_team_id) を upsert キーにすると、**延期で game_date が変わった瞬間に別レコードとして挿入され、旧行が SCHEDULED のまま残り続ける。**

- status に POSTPONED を追加する (中止と延期は予測の扱いが異なる)

- `rescheduled_to` で延期先を指す
- `result_revision` を持ち、スコア訂正時に `prediction_results` の再評価をトリガする
- 全ファクトテーブルに `updated_at` を持つ（`fetchd_at` は「取得時刻」であって「値が変わった時刻」ではない）

集計層

`accuracy_summary` を日次で洗い替える。`/api/v1/accuracy` は overall / bySeason / byModel / calibration / byProvisional の5系統を返すが、`prediction_results` に `season_id` と予測確率がないと毎回3テーブルの全件走査になる。日次バッチが既に走っているのだから、そこで量んでおく。

チーム目標値にも恒等式を課す

`predictions` はチームレベルの目標値（試投数・成功率・リバウンド等）を保持するが、**これらは個別に保存できる値ではなく、相互に制約を満たさなければならない。**

制約	内容
成功率 ≤ 試投数	試投数と成功率（0～1）の積で持てば構造的に成立する
得点 = 2FGM×2 + 3FGM×3 + FTM	得点を独立に保存せず、恒等式で導出する
FGA = 2FGA + 3FGA	FG を独立に保存しない

この制約を課さないと整合化が原理的に成立しない。各項目を独立に生成すると約2%の確率で `FT成功率 > FT試投数` のような実行不能な目標が生まれ、どんなアルゴリズムでもその目標には到達できない。DB の CHECK 制約と `test_team_targets_are_feasible`（A-16）の両方で担保する。

子テーブルの凍結は例外を設けない

`is_final = 1` による凍結は `predictions` だけでなく、**`player_predictions` と `prediction_reasons` にも同じ強さで適用する。**

旧版は `predictions` にのみトリガを置いていたが、それでは「勝率は変えられないが、個人スタッツと根拠は後から書き換えられる」状態になる。根拠は「なぜその予測になったか」の記録であり、試合後に差し替えられるなら予測の不変性という主張そのものが成立しない。

- 親の `is_final = 1` に紐づく子行の UPDATE / DELETE をトリガで禁止する
- **例外は設けない。**表示上の誤り（ラベルの誤字など）が見つかった場合も、過去の行は修正せず、以後の生成ロジックのみを直す
- 再推論による追加は、親が `is_final = 0` の場合に限り、親の非活性化と同一 `batch()` の中でのみ許される

4. 処理設計

4.1 バッチジョブ一覧

ジョブ	スケジュール	処理
<code>daily_ingest</code>	21:00 UTC（06:00 JST）	前日結果取得、照合、 スナップショット更新 、Elo再計算、推論、静的JSON書き出し
<code>gameday_update</code>	固定 cron 3スロット （02:00 / 04:00 / 07:00 UTC = 11:00 / 13:00 / 16:00 JST）。各スロットで対象を絞り込む	エントリー取得、再推論、静的JSON更新
<code>finalize</code>	毎時（Workers Cron Trigger）	開始時刻を過ぎた予測の freeze
<code>monthly_train</code>	毎月1日 19:00 UTC（= JST 2日04:00）	全データ再学習、同一ウィンドウで検証（タイムアウト120分。選手モデルは16本あり fold 数を3に制限）
<code>parser_canary</code>	日次	日程ページ1枚のみ取得してパース。失敗で Actions を fail
<code>backfill</code>	手動	過去シーズンの一括取得（再開可能）

`gameday_update` は「固定 cron + 対象の絞り込み」で実現する。旧版の 17:00 JST 単発では、土日2連戦で常態化している日曜14:05開始の試合が**すでに終了した後に再推論**される。

ただし GitHub Actions の cron は**動的に起動時刻を変えられない**（`schedule` は静的な記述であり、ワークフロー自身が次回実行時刻を指定する手段がない）。旧版の「当日の最小 `tipoff_at` を読んで N 時間前に動的起動する」という記述は実現不能だった。

代わりに次の構成をとる。

1. **固定 cron で1日3スロット起動する**（11:00 / 13:00 / 16:00 JST）。B.LEAGUE の開始時刻は概ね 14:05 / 15:05 / 17:05 / 19:05 に集中するため、このスロットで大半の試合の「2～3時間前」を捕捉できる
2. 各スロットの先頭で当日の試合を読み、**対象を `tipoff_at > now + 30分` の試合に限定する**。すでに開始した試合・開始直前の試合は処理しない
3. 当日に試合がなければ、スクレイピングを行わずに即座に終了する（外部アクセスを発生させない）
4. cron 遅延で `tipoff_at` を過ぎていた場合、Workers 側の tipoff ガードが 409 を返す。**遅延しても壊れない**

スロット数を増やすほど捕捉率は上がるが、スクレイピング回数とワークフロー実行数が増える。3スロットは「試合のある日に1日3回まで」という取得ポリシー（要件5.2）と整合する。

`finalize` を `daily_ingest` から切り出す。旧版では翌朝06:00の `daily_ingest` 内で実行され、試合開始から約11時間 freeze されない状態が毎日生じていた。しかも外部スクレイピングの後段にあるため、429 で中止されると freeze ごと巻き添えになる。freeze は外部アクセスを必要としない純粋な D1 操作なので、独立させる。

cron は UTC で記述する。JST から9時間引く。日付をまたぐ場合は曜日指定もずらす。GitHub Actions の cron は数十分～数時間遅延しうするため、遅延しても壊れない設計（tipoff ガード）を前提とする。

4.2 日次取り込みの流れ

```
1. robots.txt / 利用規約のハッシュ照合 → 変更があれば中止
2. 前日の試合一覧とボックススコアを取得（3秒間隔・値域検証）
3. games / team_games / team_game_stats / player_game_stats を upsert
   → 同一データから batch/snapshot/*.parquet を書き出し MANIFEST を更新
4. スコア訂正を検出したら result_revision を進め、再評価キューに載せる
5. 確定予測と実績を照合 → prediction_results 登録（中止・延期は除外）
6. accuracy_summary を洗い替え
7. Elo・各種レーティングを再計算（対象期間を洗い替え）
8. 向こう7日間の試合について特徴量を生成（**スナップショットを参照。D1 は読まない**）
9. 有効なモデルで推論 → predictions 追記（旧行を is_active=0）
10. SHAP をグループ集約 → prediction_reasons 登録
11. 静的JSON（today.json ほか）を書き出し、Pages へ反映
12. ingestion_logs に結果を記録
```

外部アクセスの失敗が内部処理を巻き添えにしないこと。429 で中止するのはステップ2のみで、ステップ5～11（照合・Elo・推論・書き出し）は実行して `PARTIAL` で終わる。旧版は try が処理全体を覆っていたため、前日結果が取れないだけでその日の予測更新が止まっていた。

各ステップは幕等とする。

4.3 エラー処理方針

事象	対応
個別試合の取得失敗	スキップしてログに記録。ジョブは継続
値域検証エラー	当該試合をスキップ。ログに記録
パース失敗（構造変更の疑い）	連続3件でジョブ中止。 <code>PARTIAL</code> で記録し exit 1 （通知に乗せる）
429 / 503	スクレイピング区間のみ中止。内部処理は継続
robots.txt 変更検知	ジョブ中止。人間の確認を待つ
D1 書き込み失敗	3回まで再試行。失敗時は異常終了
推論失敗	前回の静的JSONを維持。画面に更新遅延を表示
学習の検証基準未達	新モデルを有効化せず、現行を継続使用

`PARTIAL` を exit 0 で終わると、GitHub Actions の失敗通知に乗らず誰も気づかないまま数日放置される。

4.4 幕等性

- 試合は公式試合IDを主キーとして upsert
- エントリー情報は取得ごとに全行を洗い替える（追加のみだと推定行が残り 暫定 が解除されない）
- 予測は追記型。同一ジョブ run 内のリトライのみ置換を許し、ジョブを跨ぐ再推論は必ず追記する
- Elo は差分更新せず、対象期間を再計算して洗い替える。`backfill` で過去データを追加した後は全期間の洗い替え（`--full`）を実行する
- 原子性が必要な操作は単一 `batch()` にまとめる: (a) 予測の非活性化+挿入、(b) `team_ratings` の期間 DELETE+INSERT、(c) `model_versions` の切替
- `ingestion_logs` のみ幕等でない（意図的）

4.5 更新遅延の判定

`stale` は「`status = 'SUCCESS'` の最新レコードの `finished_at` から24時間以上経過」と定義する。

旧版の「最終取り込みから24時間」では、ジョブが毎日失敗しても最新レコードの時刻は毎日更新されるため、`stale = false` のまま古い予測が「正常」として表示され続けた。24時間監視をしない前提だからこそ、この自己申告が唯一の検知手段になる。

`PARTIAL` / `ABORTED` / `FAILED` が直近にあれば別バッジを出す。`gameday_update` は試合がある日のみを判定対象にする。

5. 画面設計

5.1 画面遷移

```
/ （今日の予測）
├─ /games/[id]（試合詳細：試合前 / 試合後）
│   └─ /teams/[slug]（チーム別）
├─ /schedule/[date]（日付別）
├─ /results（結果）
├─ /accuracy（的中率）
└─ /about（免責・出典・個人情報） ← 全ページ共通フッターから到達
```

5.2 主要画面の構成

今日の予測（ / ）

<ブランド>

[テーマ]

[更新遅延バナー（該当時のみ）]

今日の予測 | 結果 | 的中率

9月22日（火）の試合

B.PREMIER 19:05

[暫定]

ホームA

68%

—

32%

アウェイA

予想スコア

84

—

78

今季的中率

68.2%

(312試合)

r

圧縮カード

ホームB

55%

—

45%

アウェイB

ほぼ互角

・

17:05

[フッター：免責・出典・about]

← 両チーム分を必ず表示

圧縮カードでも両チームの勝率を表示する。片側だけだと誰の55%か判別できず、要件8.3に違反する。45～55%は「ほぼ互角」と表現し、一方的な試合と見た目で区別する。

試合詳細・試合前（ /games/[id] ）

- 対戦と勝率（両チーム、大きく表示）
- 予想スコア（整数）
- 状態表示（暫定 / 序盤）と、その理由の一文
- なぜこの予測になったか — 要因グループ別、有利な側を日本語で明示
- 個人スタッツ予測 — 出場確率50%以上の選手。**段階開示**（初期は MIN / PTS / TR / AS、行をタップでフルボックススコアを展開）。誤差の目安は主要4項目に併記
- 両チームの直近成績
- 使用モデルのバージョンと、そのモデルの通算精度（母数つき）

試合詳細・試合後

このプロダクトで最も信頼が揺れる場面であり、専用の構成を持つ。

- 実際のスコアと勝敗
- 予測との対比
- この予測の位置づけ

文言例（的中時）：「予測どおりでした（予想 84-78 / 実際 88-81・得点差の誤差 3点）」

文言例（外れた時）：「この試合は予測を外しました。ホーム68%と予想しましたが、アウェイが勝ちました。**68%と予想した試合は、これまで42試合中29試合（69.0%）が的中しています。**」

外れを認めた上で、較正が取れていること自体を信頼の材料として示す。外れた試合に強い否定色を使わない。

的中率（ /accuracy ）

一般向けから専門向けへ、この順で並べる。

- 「312試合中 223試合を的中（71.4%）。『ホームが必ず勝つ』と予想した場合は 60.1%でした。」 — ベースライン比較を最初に置く
- シーズン別の推移
- 較正の言い換え（表）：「70%くらいと予想した試合 88件 → 実際に勝ったのは 63.6%。予想よりやや勝てていません。」
- 較正曲線（グラフ） — 軸は日本語。対角線に「この線に近いほど、数字どおりに当たっている」と注記
- 暫定/確定の内訳：「出場選手が未発表の段階で出した予測 68.0%（120試合）／確定後 73.0%（192試合）」
- 詳しい指標（折りたたみ） — Brier / Log Loss。「0に近いほど良い指標です」を併記

Brier をトップ画面に出さない。0.186 のような小数点前ゼロ省略は使わず 0.186 と書く。

このサイトについて（ /about ）

非公式表明 / データ出典 / 予測の仕組み / 免責 / 個人情報の取扱い / 連絡先

5.3 表示ルール

状態	表示	条件
暫定	バッジ「暫定」＋詳細で理由	出場者未確定
確定	「確定」	試合開始後
序盤	「両チームとも今季N試合しか消化していないため…」	消化試合5未満
更新遅延	全幅バナー「表示中の予測は…時点のものです」	SUCCESS から24時間以上
試合なし	「N月N日に予定されている試合はありません」＋次の試合日	当日に試合がない
オフシーズン	「今シーズンの全日程が終了しました」＋的中率への導線	シーズン外
予測なし	「この試合の予測はまだありません。試合前日の朝に公開されます。」	予測レコードなし
エラー	「予測を読み込めませんでした。」	取得失敗

6. デザインシステム

6.1 デザイントークン

トークン	ライト	ダーク	備考
--bg	#F6F7F9	#0B0E13	
--surface	FFFFFF	#131820	
--border	#E5E8ED	#212A36	
--text	#0F1622	#E6EAF2	
--text-2	#5A6577	#93A0B5	
--text-3	#667184	#7C8798	本文用。4.9:1 以上
--text-4	#8E99AB	#59657A	野線など非テキスト専用
--accent	#416A0E	#A3E635	ライトは小さな文字でも 4.5:1
--accent-bg	rgba(65,106,14,.10)	rgba(163,230,53,.12)	
--track	#E8EBF0	#232C3A	
--warn	#7A4A12	#E0A458	注意状態の文字
--warn-bg	#FEF3E7	rgba(224,164,88,.12)	

旧版の `--text-3` は実測でライト 2.88:1 / ダーク 3.03:1 しかなく、基準を満たしていなかった。しかもこの色に的中率・セクション見出し・敗者側の確率を割り当てており、**最も守るべき情報が最も読めない色**に置かれていた。

敗者側の確率には `--text-3` ではなく `--text-2` を使う。勝敗の差は色の濃淡ではなくバーの幅と数値で伝える。

コントラスト検証をCIのトークン単体テストに組み込む。目視のスナップショット確認では必ず見落とす。

6.2 タイポグラフィ

用途	サイズ	ウェイト
勝率（主役）	36px	800
予想スコア	28px	800
見出し	19px	800
本文	13-14px	400-600
ラベル	10-11px	700-800、字間を広げる

フォントはシステムフォントスタックを既定とする。日本語 Web フォントを4ウェイト読むと初期表示分だけで数百KB～1MB級になり、4GでのLCP 2.5秒が成立しない。Web フォントは**数字とラテン文字のみ**を `unicode-range` で切り出した**1ウェイト**（60KB以内）に限る。`font-display: swap` と `size-adjust` でCLSを抑える。

ロゴ・選手写真を使わないためLCP要素はテキストになる。この優位をフォントで潰さない。

すべての数値に `font-variant-numeric: tabular-nums` を適用する。

6.3 レイアウト原則

- モバイルファースト。コンテンツ幅の上限は 440px 程度
- 左右 16px のガター。375px で横スクロールを発生させない
- カードの角丸 14-16px、境界線 1px
- ライトでは境界線＋淡い影、ダークでは境界線のみ

6.4 チャート

ライブラリを使わずインライン SVG で実装する。表示するのはシーズン数十点と校正バケット5〜10点で、Recharts や Chart.js を入れると `/accuracy` だけで gzip 後100〜300KB 増える。デザイントークンによる配色統一とも相性がよい。

7. インフラ・デプロイ設計

7.1 リポジトリ構成

```
bpredict/
├─ CLAUDE.md
├─ docs/
├─ web/                Next.js（静的出力）
│   └─ public/data/    バッチが書き出す予測JSON
├─ api/                Cloudflare Workers
├─ batch/              Python
│   └─ tests/fixtures/ 合成HTML（実サイト由来の文言を含まない）
├─ db/
│   └─ migrations/
│   └─ seeds/          マスタ初期データ / 開発用シード
└─ .github/workflows/
    ├── ci.yml          push / PR
    ├── parser-canary.yml  日次
    └─ （バッチ各種）
```

7.2 CI

push / PR で必ず実行する。最重要の `test_leakage.py` を人間の規律に任せると、3ヶ月で形骸化する。

対象	内容
Python	<code>ruff / mypy / pytest batch/tests</code> （リークテストを含む）
api	<code>tsc --noEmit / eslint / vitest</code> （Zod契約、認証、キャッシュヘッダ）
web	<code>tsc --noEmit / eslint .</code> （ <code>next lint</code> は Next.js 16 で削除された） / <code>next build</code> / トークンのコントラスト検証 / <code>out/</code> のファイル数検査（18,000以下）
リポジトリ	fixtures に実サイト由来文字列がないことの検査 / スナップショットと D1 ローカルの一致検査

`next lint` を使わない。Next.js 16 で削除されており、`next build` もリントを実行しなくなった。ESLint は CLI を直接呼ぶ（`eslint .`）。設定は Flat Config（`eslint.config.mjs`）とする。`next lint` を CI に書くとコマンドが存在せず失敗するか、将来の互換シムに依存することになる。

`monthly_train` の先頭にもリークテストを置き、失敗したらモデル登録に到達させない。

`parser_canary` は日次で実サイトの日程ページ1枚のみを取得してパースし、必須項目が取れなければ Actions を fail させる。fixture はスナップショットであるため、定義上サイト構造の変更を検知できない。これが唯一の早期検知手段になる。

7.3 Actions のセキュリティ

項目	設定
<code>permissions</code>	既定は <code>contents: read</code> 。 <code>daily_ingest</code> と <code>gameday_update</code> の2つのみ <code>contents: write</code> （静的JSONとスナップショットをコミットするため）。それ以外に <code>write</code> を与えない
リポジトリ設定	Workflow permissions を「Read repository contents」既定に
サードパーティ Action	増やさない。増やす場合はコミットSHAでピン留め
<code>pull_request_target</code>	使用禁止
Dependabot	<code>pip / npm / github-actions</code> を weekly で登録
Secret scanning	Push protection を有効化（ <code>git grep</code> と違い push 時点で止まる）
依存固定	<code>requirements.txt</code> は <code>==</code> 固定、 <code>package-lock.json</code> をコミットし <code>npm ci</code> を使う

7.4 シークレット管理

名称	保管先	用途
<code>INGEST_TOKEN / INGEST_TOKEN_NEXT</code>	GitHub Secrets + Workers Secret	内部API認証（2キー方式）
<code>FINALIZE_TOKEN</code>	同上	freeze 専用（破壊的操作を分離）
<code>CF_API_TOKEN</code>	GitHub Secrets	マイグレーション専用。スコープは対象D1のEditのみ

`D1_DATABASE_ID` は機密ではない（操作には API トークンが別途必要）ため `wrangler.toml` に直書きしてよい。環境変数注入は `d1_databases` の必須フィールドでは効かないため、旧版の「直書きしない」という制約は実現不能だった。

トークンの無停止回転手順

- 新トークンを Workers Secret に `INGEST_TOKEN_NEXT` として追加

- 2. 認証は現行・次期のいずれかに一致すれば通す
- 3. GitHub Secrets を更新
- 4. 次のパッチ成功を確認
- 5. 旧トークンを削除

例外オブジェクトをそのままログ・DBに入れない。D1 REST API の URL にはアカウントIDとDB IDが含まれるため、`str(e)` を `ingestion_logs.error_message` に保存し `/api/v1/health` で公開すると、それらが公開APIから漏れる。例外は型名と自前の短いメッセージに限定し、`/health` は `error_message` を返さない。

7.5 無料枠の監視

リソース	無料枠	想定使用量
Workers リクエスト	10万per日	数百〜数千per日（主要導線が静的配信のため）
Workers CPU	10ms per リクエスト	実測 1.34ms（500行の JSON パース+検証）。Zod で3〜4ms 見込み
Workers サブリクエスト	50 per リクエスト	0（D1 Binding はサブリクエストに数えない）
D1 読取行	500万per日	約1万per日（学習はスナップショット参照のためD1を読まない）
D1 書込行	10万per日	backfill 1シーズンで約26,500行
D1 クエリ数	50 per Worker 呼び出し（Free）	設計上の真の制約。1リクエストあたり行数をテーブルごとに算出する（詳細設計 3.4）
D1 バインドパラメータ	100 per 文	<code>max_rows_per_request = floor(100 / 列数) × 40</code>
D1 1行のサイズ	2,000,000 バイト	<code>artifact_text</code> に1.5MB の上限チェックを課す
D1 ストレージ	5GB	100MB未満
Pages ファイル数	20,000 per デプロイ	約10,184（直近5シーズン、51%）
Pages ファイルサイズ	25 MiB per ファイル	静的JSONは1日分30KB程度
Pages ビルド	500回per月	約90回per月
WAF レートリミット（Free）	1ルール / カウント期間10秒のみ / ブロック10秒のみ / IP のみ / 対象フィールドは Path と Verified Bot のみ	<code>/api/v1/*</code> に1ルール
GitHub Actions	public のため無制限	月 約600分

設計を実際に拘束するのは、ストレージでも CPU でもなく「D1 の1呼び出し50クエリ」である。検証の結果、9テーブル中6つが旧設計のパッチサイズ（500行）でこの上限を超えていた。 `player_predictions`（26列）は500件で167文を要し、上限の3倍を超える。

Workers CPU 10ms は当初 Critical と判断したが、実測で否定された。500行の JSON（141KB）のパース 0.56ms、検証 0.28ms、batch 組立 0.50ms で合計 1.34ms。Zod は手書き検証より数倍遅いため実際は3〜4ms と見込まれるが、それでも予算内に収まる。パッチサイズはCPUではなくクエリ数で決める。

batch() 内の各文が50クエリ制限にどう計上されるかは公式に記載がない。Phase 0（P0-13）で実アカウントでの実測が必要であり、それまでは「1文＝1クエリ」という最も厳しい前提で設計する。

backfill は1日1シーズンに限る。

`/api/v1/health` に当日のカウンタ（Workers リクエスト数、D1 rows_read 積算、キャッシュヒット率、前回パッチ所要時間）を出し、A-10 の検証手段とする。

8. 品質保証

8.1 テスト方針

層	配分	内容
単体	約70%	特徴量・Elo・パーサ・整形
結合	約25%	D1ローカルでのAPIテスト、パッチの二度流し
E2E	数本	テーマ保持、横スクロールなし、免責への到達性

画面スナップショットは3画面×2テーマから、1画面×2テーマ+トークンのコントラスト単体テストに縮退させる。色トークンを1つ変えるたびに6枚落ちる維持コストは個人開発に見合わない。A-06（再訪時のテーマ保持）とA-07（免責への到達性）はスナップショットでは原理的に検証できないためE2Eで行う。

テストデータ: 個人情報を持たないため本番コピーも匿名化も不要。db/seeds/test/ に架空8チーム×2シーズン（約300試合）の決定論的シードを置く。实在選手名を固定しない。pytest のスキーマは db/migrations/*.sql をそのまま in-memory SQLite に適用する（二重管理するとテストが本番を反映しなくなる）。

8.2 リーク検証（最重要）

検証の向きは「as_of をずらす」ではなく「DBを攪乱する」。

旧版のテストは as_of を7日後にずらして値が変わらないことを assert していたが、これは論理が逆だった。as_of を後ろにずらせば参照できる過去試合が増えるため、正しい実装なら値は変わるのが正常である。あのテストは正しい実装を落とし、as_of を一切見ない実装だけを通していた。

テスト	内容
test_target_game_mutation_does_not_change_features	as_of 固定のまま対象試合のスコア・スタッツ・観客数を攪乱し、全キーが不変であること（本命）
test_future_game_mutation_does_not_change_features	as_of 以降の試合を攪乱し、不変であること
test_as_of_is_actually_applied	as_of を前にずらすと結果依存の特徴量に変化すること（引数が効いている陽性確認）
test_backtest_uses_production_data_as_of	保存済み feature_snapshot と再構築結果が一致すること
test_features_exclude_unfinished_games	SCHEDULED の試合が集計に入らないこと
test_features_exclude_in_progress_game	開始済み・未終了の試合が入らないこと
test_accuracy_within_plausible_band	baseline < accuracy < 0.85 の両側境界
ミューテーション試験	意図的にリークさせた実装でテストが落ちることをCIで確認（テストのテスト）

sql_spy はクエリ文字列だけでなくバインドパラメータも記録すること。プレースホルダを使えば game_id はクエリ文字列に現れず、文字列一致では検出できない。

8.3 その他の必須テスト

分類	代表的なケース
freeze	test_finalize_at_tipoff_boundary（前後1秒）、test_internal_predictions_rejects_after_tipoff、test_reinference_appends_not_replaces、test_cron_delay_simulation
冪等性	test_daily_ingest_twice_produces_identical_db（全テーブルのハッシュ一致）、test_evaluate_is_idempotent、test_recompute_ratings_full_rebuild
境界・異常系	ゼロ除算（possessions NULL、fga = 0、capacity NULL）、POSTPONED、シーズン跨ぎの rest_days、新規参入クラブ、全選手欠場、確率の合計、"MM:SS" / "DNP" / 全角数字のバース
整合性	test_win_prob_and_score_agree、test_win_probs_sum_to_one_after_calibration、test_only_one_active_model_per_type、test_player_predictions_reconcile_to_team（得点 ±0.5点・その他 ±2%・総出場時間200分）、test_player_prediction_identities（FGM = 2FGM + 3FGM、PTS = 2FGM×2 + 3FGM×3 + FTM、成功数 ≤ 試投数）
整合化	test_reconciliation_converges（1,500ケースのランダム入力で3回反復後の項目誤差99%点が2%以内、得点誤差が±0.5点以内、制約違反0件）、test_reconciliation_no_clipping（成功率が [0,1] を出ないこと）、test_team_targets_are_feasible（チーム目標が恒等式と 成功数 ≤ 試投数 を満たすこと）、test_minutes_sum_to_200
凍結の網羅	test_player_predictions_frozen_when_parent_final、test_prediction_reasons_frozen_when_parent_final（子テーブルにも例外なくトリガが効くこと）
スナップショット	test_snapshot_matches_d1（行数と主キー集合の一致）、test_snapshot_manifest_hashes、test_training_reads_no_d1（学習パスでD1クライアントが呼ばれないこと）
無料枠	test_batch_size_within_query_limit（各テーブルの max_rows_per_request が floor(100/列数)×40 以下で、1リクエストの文数が50以下）、test_artifact_size_guard（1.5MB 超の artifact が拒否されること）
回帰	test_golden_predictions_within_tolerance（代表100試合の予測変化量）、test_feature_list_diff_is_reported（欠損率30%超で不採用）

8.4 運用監視

- /api/v1/health にジョブ別の最終成功時刻と当日カウンタを公開
- PARTIAL は exit 1 で終え、Actions の失敗通知に乗せる
- 直近50試合の Brier が通算より 0.03 以上悪化したら degraded を立て、日次ジョブで検出

9. 未決事項

#	事項	決定期限
U-01	プロダクト正式名称（商標調査を含む）	Phase 0
U-02	入場者数が公式サイトから取得可能か	Phase 0
U-03	ボゼッション算出に必要な項目が揃うか。係数0.44の妥当性	Phase 0
U-04	旧B1データと新リーグのクラブ対応関係	Phase 0
U-05	個人スタッツ予測の表示対象人数	Phase 0（A-01の判定に必要）
U-06	エントリー情報の公開有無と確定タイミング	Phase 0
U-07	1試合あたりの取得ページ数と backfill の所要日数	Phase 0
U-08	統計データの有償提供の有無（30条の4ただし書の判断材料）	Phase 0

U-05 と U-06 は受け入れ基準 A-01 / A-02 の判定条件そのものであり、未決のままでは品質ゲートが機能しない。

9.1 実装前に実測が必要な項目

設計段階で数値検証を試みたが、サンドボックスの制約（PyPI / npm への到達不可）により実行できなかった項目。これらは「未決事項」ではなく「値が未測定の確定事項」であり、設計は最も厳しい前提で書いてある。実測の結果が前提より緩ければ制約を緩められるが、実測前に緩めない。

#	項目	現在の前提	実測で変わりうること
P0-11	勝率の導出経路 A / B の比較	両方実装	どちらを採用するか
P0-12	LightGBM の実 artifact サイズ	書式からの推定 0.27~0.70MB	1.5MB を超えるなら gzip+base64 の追加
P0-13	D1 batch() のクエリ計上	1文=1クエリ (最も厳しい)	緩ければバッチサイズを上げられる
P0-14	Next.js 16 の実ビルドファイル数	1ルート2ファイル	多ければ静的生成を3シーズンへ縮小
P0-15	Zod の実検証コスト	手書き検証の3~4倍 (3~4ms)	10ms を超えるなら手書き検証へ
P0-16	ECE ノイズフロアの実分布での値	一様分布での参考値	閾値の確定

詳細は docs/requirements.md 11章。設計段階で実行できた9件の検証結果とスクリプトは verification/ にある。